Activity 5 Report: Database Maintenance and Monitoring

COMP 018 Database Administration | Project tested: BrewPOS (Cafe Point-of-Sale) (Spring Boot + PostgreSQL) | Date of testing: 6 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	BrewPOS (Cafe Point-of-Sale)
Owning group	Group 4: Olivarez, Charles Kean E.; Infantado, Gerwin G.; Pocsidio, Jhon Louie F.; Abangan, SJ Kaizen M.; Bartolata, Redge Raindel N.
Repository	https://github.com/Nycri/brewpos
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a5_health.txt, a5_logs.txt, a5_maintenance.txt

This is a full re-test. The database was dropped and rebuilt from the repository files, and every result below comes from this run.

Run at 00:03 on 7 November 2026 (Asia/Manila) after the application and access tests, with the app connected as brewpos_app.

2. Health and activity

Check	Result
Version	PostgreSQL 18.6, x86_64 Windows
Database size	8406 kB
Connections	11 backends: 10 from the app pool (brewpos_app) and the monitoring session. max_connections is 100
Transactions	439 commits, 52 rollbacks (from the permission and constraint tests)
Cache	57,889 buffer hits against 9 reads, about 99.98% hit ratio
Deadlocks and temp files	0 and 0
Autovacuum	on, but no table had been vacuumed or auto-vacuumed yet
Rows	menu_category 5, menu_item 15, order_line_item 16, order_ticket 11, staff 2
Table size	40 kB per table (24 kB for staff), mostly index pages

Dead tuples before maintenance: menu_item 21, menu_category 12, order_ticket 6, order_line_item 5. They come from the price updates and deleted test rows and were below the autovacuum threshold, so autovacuum had not acted.

3. Logging

Setting	Value	Comment
log_statement	none	No statement logging
log_min_duration_statement	-1	Slow queries are not logged
Server log folder	C:/Program Files/PostgreSQL/18/data/log	Readable
Application log	Console only	Spring Boot wrote no log file; the console output had no WARN or ERROR lines

4. Log findings

The PostgreSQL log is shared by everything on this server, so older entries from other projects tested earlier the same evening also appear in it. Only entries from 23:54 onward belong to BrewPOS.

Time	Entry	Explanation
23:57:02	Check constraint violations (price, quantity, payment, staff role) and two foreign key violations	The direct constraint tests of Activity 3. The API-level rejections do not appear here because the app answers them before the database
23:59:55 to 23:59:56	permission denied for table staff, order_ticket, order_line_item, schema public; must be owner; permission denied to create role; permission denied for pg_authid; FK on menu_category	The brewpos_app tests of Activity 4, all expected
00:00:26	canceling statement due to statement timeout, and FATAL permission denied for database "postgres"	The 31-second sleep test and the cross-database connection test
00:00:27 to 00:00:28	permission denied for table order_ticket, order_line_item and a sequence	The group role tests
00:03:22	permission denied to vacuum and to analyze "order_ticket", skipping it	brewpos_app trying maintenance (section 5)

No unexplained errors. The application console log contained no warnings or errors during the whole session.

5. Maintenance performed

Action	Result
ANALYZE (whole database)	Completed, last_analyze updated on all 5 tables
VACUUM (VERBOSE, ANALYZE) menu_item	21 dead row versions removed, 15 remain, 1 page, index scans 1
VACUUM (VERBOSE, ANALYZE) order_ticket	6 dead row versions removed, 11 remain
Dead tuples afterwards	menu_item 0, order_ticket 0 (menu_category 12 and order_line_item 5 left for autovacuum)
VACUUM and ANALYZE as brewpos_app	Skipped: permission denied (not the table owner). The project defines no maintenance role

6. Query plans

The three queries below are the ones the app runs for the Orders list, Daily Sales and best sellers. They were run with EXPLAIN (ANALYZE, BUFFERS).

Query	Plan	Planning / execution
Orders list newest first	Hash join of order_ticket and staff, then sort	2.66 ms / 0.08 ms
Daily Sales for today	Sequential scans with a hash join and aggregate	3.23 ms / 0.08 ms
Best sellers by item	Hash join and hash aggregate	3.04 ms / 0.10 ms

With only 11 orders a sequential scan is the right plan. To see what happens as the table grows, sequential scans were switched off for one transaction (and rolled back). The Daily Sales query then used idx_order_ticket_timestamp and the order_line_item primary key, so the index is usable for the date filter once the table is large enough for the planner to prefer it. All 9 indexes have been used by the test traffic.

7. Recommendations

- Create a maintenance role (member of pg_maintain) and run scheduled VACUUM and ANALYZE with it, because brewpos_app cannot.
- Set log_min_duration_statement to about 500 ms, and log_statement to ddl.
- Give the application a log file (logging.file.name) with rotation. At the moment diagnostics only exist in a console window.
- Add a limit on order quantity (see Activity 3), otherwise a single bad order skews every report.
- Re-run EXPLAIN on Daily Sales when there are tens of thousands of orders.

8. Feedback for the owners

No health problems found. The useful items are the maintenance role, slow-query logging and an application log file.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.